

Soft Actor-Critic for Dexterous Humanoid Hand Grasping on Complex Climbing Holds

Lea Lestál (leal@itu.dk)

Course code: KIREPRO1PE

Examination group: S25KIREPRO1PE068

1 Introduction

Physics-based controllers and reinforcement learning have been used to simulate plausible climbing movements of humanoid robots, however I found no research on climbing with realistic hands. Previous work in this topic has achieved smooth, multi-limb climbing movements, using spherical holds and wall friction (Kang et al., 2025). However the hold positions are relatively simple, without simulating fingers or more complex holds with geometries. Most simulation projects have very low-fidelity contact: binary “grip/no-grip” point contacts. With a low-fidelity grip, the policy may rely on the simplified contact model, whereas a more realistic grip could lead to more plausible climbing movements.

The focus of this project was, to extend the existing work with more complex hands and hold geometries, investigating how contact model fidelity affects the Soft Actor-Critic (SAC) policy. For this project I relied heavily on the work of Zakka et al. (2023), the RoboPianist, and I used their implementation of the Soft Actor-Critic (SAC) algorithm.

2 Existing work

My main source was the RoboPianist. In the project Zakka et al. used their own version of SAC, for a human-like hand motion for playing the piano. They used the MuJoCo environment, and trained the dexterous hand from MuJoCo Menagerie, for 150 different songs. They aimed for smooth human-like finger usage, so they designed a reward function that doesn't solely optimize for pressing key notes, but uses prior information, stating which fingers should press a specific key in a specific moment. This helped to solve the sparse reward problem they faced.

3 Environment

The environment is based on the RoboPianist framework, extended with new models and a grasping task. The source code for the environment and learning implementation is available on GitHub (see Appendix A). I added a solid wall and a climbing hold. I modified one of the Shadow dexterous hands from MuJoCo Menagerie, I added weight, and a freejoint to the root element of the hand. I enabled gravity in the arena, to allow the hand to fall down. I added a climbing hold with really high friction, to help learning, and placed the hand directly above the hold (Figure 1).

During training, I used a jug hold which is a large shape that allows the hand to wrap around it. After training, the learned policy is tested with differently shaped unseen holds.

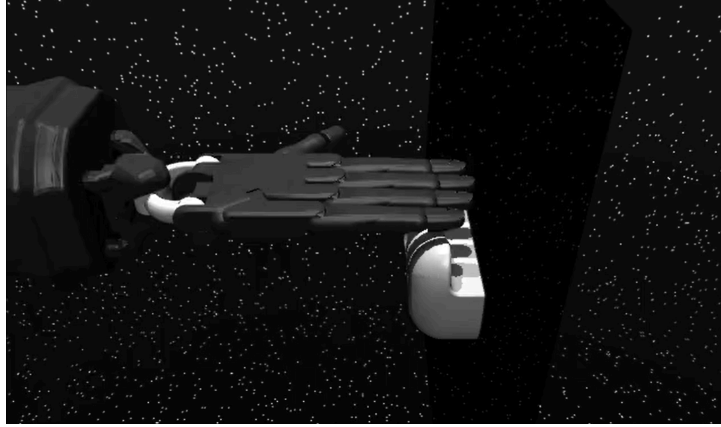


Figure 1: Initial position of the hand model adapted from RoboPianist (Zakka et al., 2023).

I model hold-grabbing as a finite-horizon *Markov Decision Process (MDP)* defined by the tuple $(S, A, \rho, p, r, \gamma, H)$. The state space $S \subset \mathbb{R}^{28}$ consists of the hand joint positions (22 dimensions), the 3D position of the hand root, and the 3D position of the climbing hold. The action space $A \subset \mathbb{R}^{22}$ represents control commands for the two wrist and twenty finger joint actuators. The state distribution ρ is deterministic, p governs the dynamics, r is the reward function. I used a discount factor $\gamma \in [0, 1)$, which is set to zero upon failure, and a fixed horizon $H = 100$ timesteps.

Actions are applied every 0.05 seconds (20 Hz). The physics simulation runs at a timestep of 0.005 seconds (200 Hz), so each action is executed over ten physics steps. The goal of the agent is to maximize the expected discounted return over the episode.

The agent's observations consist of three types of observations. The position of the hand's root element, the position of the climbing hold, and all the joint positions of the hand, which include 2 forearm and 20 finger joints.

4 Learning

I used the learning algorithm described in the RoboPianist. Zakka et al. used DroQ learning for the critic. The Q functions are 3-layer MLPs. The actor is also a 3-layered MLP and is implemented as a tanh diagonal-Gaussian. The biggest difference from the RoboPianist is that I had to reduce exploration. I also faced a similar sparse reward issue that Zakka et al. did.

I listed the hyperparameters I used, in Figure 2, with the hyperparameter used in RoboPianist, for comparison.

Hyperparameter	Value	Value in RoboPianist
Total train steps	50,000/ 500,000	5M
Optimizer		
Type	Adam	Adam
Learning rate (Actor)	3×10^{-5}	3×10^{-4}
Learning rate (Critic)	1×10^{-4}	3×10^{-4}
Learning rate (Temperature)	3×10^{-5}	3×10^{-4}
β_1	0.9	0.9
β_2	0.999	0.999
Critic		
Hidden units	256	256
Hidden layers	3	3
Non-linearity	GELU	GELU
Actor		
Hidden units	256	256
Hidden layers	3	3
Non-linearity	GELU	ReLU
Misc		
Discount factor	1	0.99
Minibatch size	256	256
Replay period	every 1 step	every 1 step
Eval period	every 10,000/ 50,000 steps	every 10,000 steps
Number of eval episodes	1	1
Replay buffer capacity	500,000	1M
Seed steps	5,000	5,000
Critic target update frequency	1	1
Actor update frequency	1	1
Critic target EMA momentum (τ)	0.005	0.005
Actor log std dev. bounds	$[-20, 2]$	$[-20, 2]$
Entropy temperature	1.0	1.0
Learnable temperature	True	True

Figure 2: Hyperparameter values compared to values in RoboPianist.

I set the learning rate for the actor very low because the environment is sensitive, and large updates might break the policy. Large actor updates often break stable grasps or cause jittery motions. A low learning rate enforces conservative policy updates, preserving good contact configurations once discovered.

The critic is trained more aggressively than the actor to accurately model complex contact dynamics and stabilize policy updates. The temperature learning rate adapts slowly to avoid rapid collapse of exploration.

In the original paper, Zakka et al. used ReLU, but they are constantly improving the codebase and changed to GELU. I decided to use GELU for non-linearity as well.

I set the replay buffer to 500,000 to encourage robustness across slightly different contact states. Initially, a successful grab was defined as the hand staying on the hold for 50 seconds, but this made learning slow and rewards sparse. I reduced this to 5 seconds, so that the episode terminates immediately with a discount of zero if the hand falls. If the episode reaches the timeout, the discount is set to 1. This puts pressure on survival but could contribute to unnatural grabs as well.

A relatively high initial entropy encourages exploration of finger placements. Making the temperature learnable allows the policy to become more deterministic once stable grasps are found, reducing unnecessary finger motion later in training.

In Figure 3 the moving average of the total reward per episode is visible for the training of 50,000 steps with a window of 200.

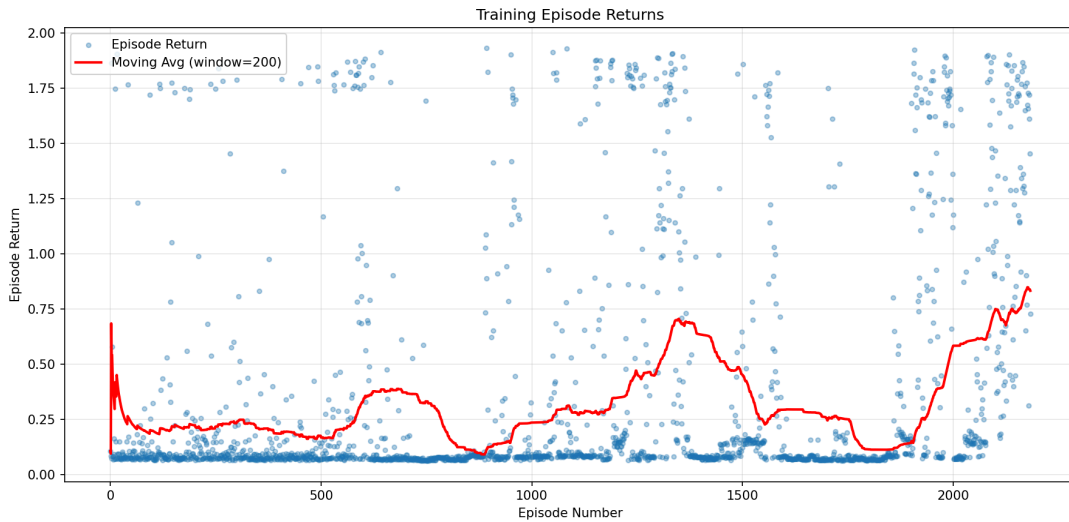


Figure 3: Moving average of returns over episodes for 50,000 steps.

5 Reward function design

I trained in several different ways, with different reward functions. My original idea was to reward the height of the hand. First, I tried a binary reward for every step the hand stayed above a certain height. Then, I added a Gaussian function to encourage it to stay around a target hand height. In both cases, I found that even though it found solutions to hold on to the grab, the hand was using weird movements, mostly using only a few fingers to grab the hold (Figure 4).

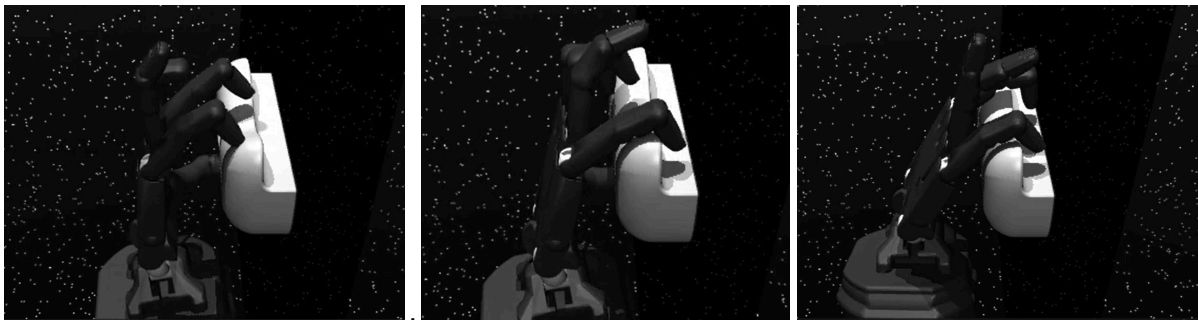


Figure 4: Unnatural grabs, where the hand uses only a few fingers.

I wanted to guide the fingers to touch the holds, so I set a normalized reward every step for each finger that touched the surface of the hold. The contact reward alone was insufficient to learn a successful grasp.

I introduced a cumulative survival reward of 0.01 for every step the hand is alive and a finger proximity reward, to bias exploration toward surfaces and to increase contact frequency.

The final reward function is the following:

$$r_{\text{survival}} = \begin{cases} 0.01 & \text{if } h_{\text{hand}} \geq h_{\text{fail}} \\ -0.01 & \text{if } h_{\text{hand}} < h_{\text{fail}} \end{cases}$$

where h_{hand} is the vertical (z) position of the hand root, and h_{fail} is 50 cm lower than the position of the jug.

$$r_{\text{finger}} = \sum_{i=1}^F \frac{1}{T_{\text{max}}} (g_i(t))$$

where F is the number of fingertips, T_{max} is the maximum episode length, $d_i(t)$ is the Euclidean distance between fingertip i and the surface of the climbing hold at time t , and $g(\cdot)$ is a Gaussian tolerance function that maps distances to values in the range $[0, 1]$.

The total reward is the sum of r_{finger} and r_{survival} .

I found that this reward function provided more realistic looking grabs than what I tried before, so I decided to test policies trained with it.

6 Experimental Observations

Videos of the experiments are available in Appendix A.

6.1 Training for 50,000 steps

I found that the model already finds a reasonably effective solution after only 50,000 steps (Figure 5).

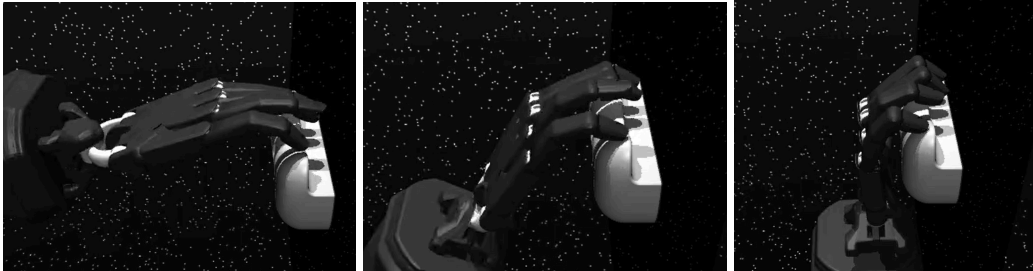


Figure 5: Human-like grab, after training for 50,000 steps.

The best policy was collected by saving checkpoints. I tested it on various hold shapes obtained from Thingiverse. The different grabs can be seen in Figure 6.

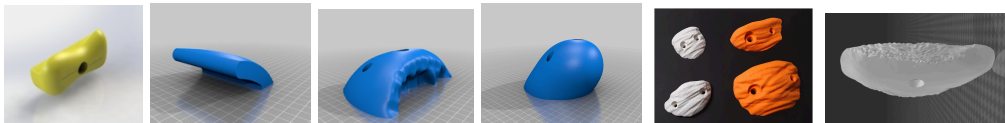


Figure 6: Jug, really easy jug, pinch-like jug, sloper, crimps and a rock-like hold, collected from Thingiverse (see Appendix B).

Jugs, which are large and easy to grasp, were tested in four variations: the base jug, a very easy jug, a pinch-like jug, and a jug turned upside down. For the last three, the policy displayed a balancing motion rather than a proper grab. Slight variations were observed: on the easy and pinch-like jugs, the hand uses the middle and pointing fingers to balance, while on the upside-down jug, only the middle finger is used (Figure 7).

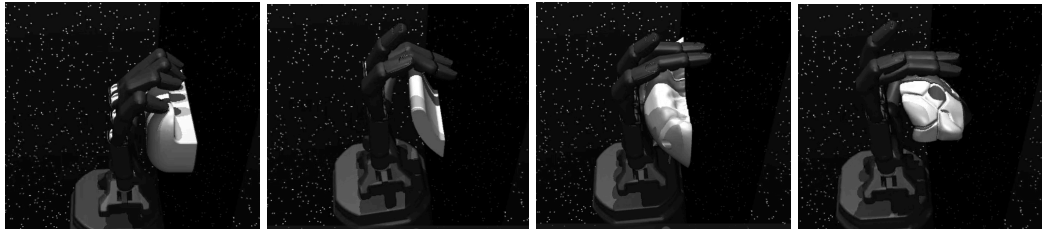


Figure 7: The hand grabbing different jugs.

Slopers are large, round and smooth without a clear edge. To grab them the climber needs to rely on body weight and friction. In test episodes, the hand fully grasps the sloper but slowly slips.

Crimps are small holds that require precise fingertip placement. I tested the policy with three crimps. With the normal sized crimp, the hand starts to balance on two fingers, and then stabilizes with the others. With the small crimp the hand grabs and rests on it. With the unevenly shaped crimp fingers are split into two groups.

I also tested with a hold that has an irregular rock-like side (Figure 8). The hand grabs with all fingers. The policy struggles when holds are positioned far below or at unusual angles.

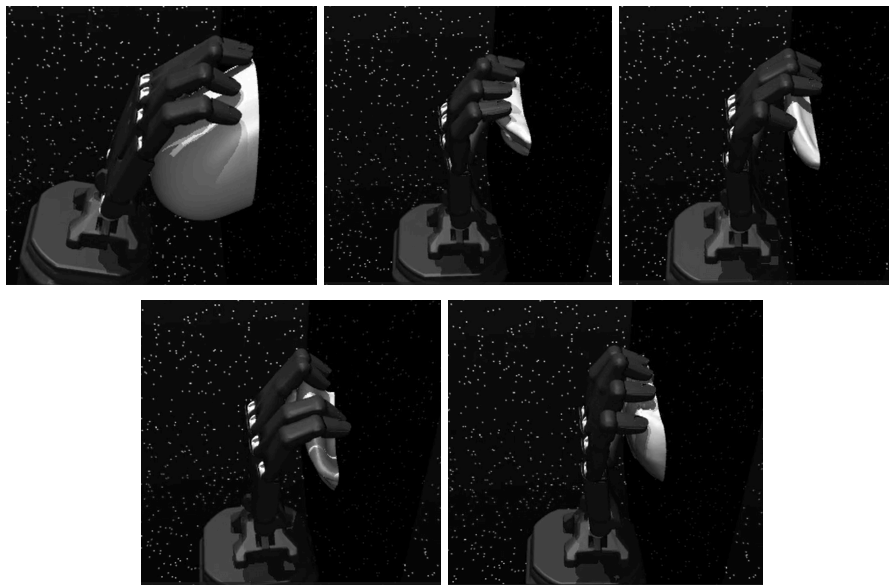


Figure 8: The hand grabbing a sloper different crimps and a rock-like hold.

6.2 Training for 500,000 steps

The second policy I experimented with was trained in 500,000 steps.

This policy achieved higher returns on the reward function, but used some less human-like movements to achieve that. The grip is a lot more stable, and the hand is able to make

contact with the hold faster. The fingers initially move a lot, and after the hand finds a stable position, it moves closer to the hold. There was also way less slipping (Figure 9).

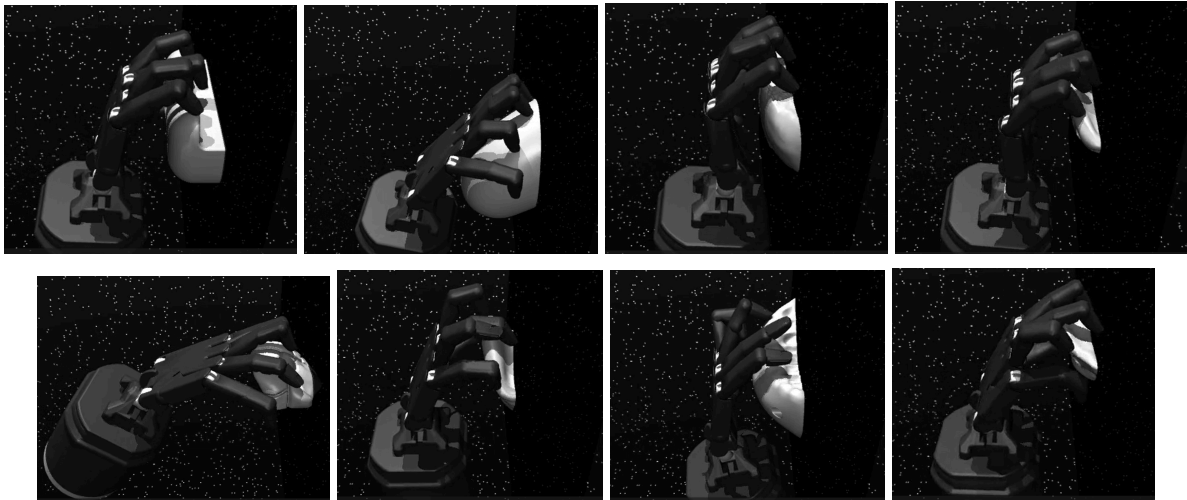


Figure 9: The hand grabbing different holds.

Despite achieving higher rewards, this policy produces less human-like behaviours, especially on crimps and pinch-like holds.

7 Discussion

Interestingly, the hand is able to grasp different types of holds (slopers, crimps, rock-like shapes) even though it was trained only on jugs. I believe this happens because, when attempting a learned grasp, it sometimes bumps into the surface of the hold. This creates a new observation for the next time step, as the joint angles and hand positions are altered. Two outcomes are possible: the hand remains stable, or it slips. If slipping occurs in a pose the policy has previously encountered and learned to respond to, the hand applies the appropriate correction. This likely explains why the hand can hold on to different types of holds.

Future work would involve making the grabbing motion more human-like. In some of the test episodes the hand is capable of holding its weight, but it finds very unnatural positions. It doesn't grab, but rather balances.

With the 500,000 step policy, other non human-like behaviours emerge. The hand makes very fast, sudden movements, and sometimes grabs the holds using the fingertips only. This can be explained by finger distance reward. Right now, it gives rewards if the very end of the finger is close to the climbing hold, and using a lower part of the underside of the fingers, could provide a much more human-like grabbing motion

Another improvement would be to grab for a longer time, or with a heavier arm (like in the real world, when a body is attached to the hand).

Another route I could follow, to achieve human-like grabs, is ensuring the fingertips are applying a useful force in a stable direction, rather than optimizing for distance.

References

Kang, J., Park, S., Hwang, I., Kim, J., & Lee, J. (2025). Learning climbing controllers for physics-based characters. *Computer Graphics Forum*, 44(1), e15284.

<https://doi.org/10.1111/cgf.15284>

Zakka, K., Wu, P., Smith, L., Gileadi, N., Howell, T., Peng, X. B., Singh, S., Tassa, Y., Florence, P., Zeng, A., & Abbeel, P. (2023). *RoboPianist: Dexterous piano playing with deep reinforcement learning*. In *Proceedings of the 7th Conference on Robot Learning* (pp. 2975–2994). Proceedings of Machine Learning Research.

<https://proceedings.mlr.press/v229/zakka23a.html>

Appendix

Appendix A: Videos and source code

Videos of the grabs are available here:

<https://drive.google.com/drive/folders/1aQT3uIPQ4NyrpaSHe-rfbvdNbUclQsF?usp=sharing>

Code is available here:

https://github.com/leakaktusz/climbing_hand.git

Appendix B: External Resources

- **Microsoft Copilot was used during the project to:**

Create scripts for plotting and visualizing data.

Assist with debugging and resolving errors in code.

- **Climbing Hold Models (Thingiverse climbing holds):**

Pinch Jug: <https://www.thingiverse.com/thing:6346817/files>

Jug: <https://www.thingiverse.com/thing:2071218>

Rock-like Hold: <https://www.thingiverse.com/thing:3514430/files>

Easier Jug: <https://www.thingiverse.com/thing:2512943>

Sloper: <https://www.thingiverse.com/thing:6338684>

Crimps: <https://www.thingiverse.com/thing:6375816>